

논문 §2.4 「격자 지도 표현과 픽셀 지목 행동」 배경 개념 해설

전결합층 · 완전 합성곱(FCN) · U-Net · CoordConv · FiLM · GAP · 포인터/어텐션

USV 군집 방어 논문 보조 자료

2026-09-20

이 문서의 목적

논문 §2.4는 한 문단에 다섯 가지 딥러닝 도구(FCN, U-Net, CoordConv, FiLM, 포인터 네트워크)를 압축해 인용한다. 이 문서는 그 도구들이 무엇이고, 합성곱의 어떤 한계를 메우며, 우리 기동 정책에서 정확히 어디에 쓰였는지를 개념을 모르는 독자도 따라올 수 있게 푼다. 각 절 끝의 「우리 논문에서는」 상자가 논문 본문(§4.4)과의 연결이다.

Contents

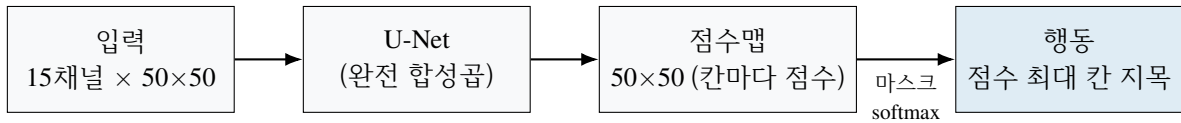
1	먼저, 문제의 모양	1
2	전결합층 — 그리고 왜 그것을 빼는가	1
3	완전 합성곱 신경망(FCN)	2
4	CoordConv — “내가 어디에 있는가”	3
5	GAP — 지도를 벡터로 요약하기	4
6	FiLM — “전장 전체가 어떤 상황인가”를 모든 위치에 알리기	5
7	포인터 네트워크와 어텐션 — “후보 중 하나를 지목”	5
8	합쳐서 읽는 §2.4	6

1 먼저, 문제의 모양

기동 계층의 정책은 다음 질문에 답한다.

“모선을 중앙에 둔 50×50 격자 전장 지도가 주어졌을 때, 방어정 p 는 어느 칸에 그물벽을 놓아야 하는가?”

입력은 이미지($15\text{채널} \times 50 \times 50$)이고, 출력은 “칸 하나”다. 이 출력 형식이 아래 모든 설계를 결정한다. 좌표 (x, y) 두 숫자를 회귀하는 대신 2,500칸 각각에 점수를 매기고 그중 하나를 고르는 방식을 택했기 때문에, 신경망은 “이미지 전체에 대한 답 하나”가 아니라 “픽셀마다 답 하나”를 내야 한다. 이것이 §2.4가 조밀 예측(dense prediction)이라 부르는 것이다.



2 전결합층 — 그리고 왜 그것을 빼는가

전결합층(fully-connected layer, FC 층, 선형층)은 가장 기본적인 신경망 층으로, 입력의 모든 값이 출력의 모든 값과 빠짐없이 연결된다. 수식은 행렬곱 하나다.

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad \mathbf{x} \in \mathbb{R}^n, \mathbf{W} \in \mathbb{R}^{m \times n}, \mathbf{y} \in \mathbb{R}^m$$

입력 n 개, 출력 m 개면 가중치가 $n \times m$ 개다. 입력 하나하나가 출력 하나하나에 각각 다른 가중치로 연결된다.

보통의 분류용 CNN은 뒤끝이 다음과 같다.

$$\text{이미지} \rightarrow \text{conv} \rightarrow \text{conv} \rightarrow \cdots \rightarrow \underbrace{\text{flatten}}_{\text{격자를 1차원 벡터로 펼침}} \rightarrow \text{전결합} \rightarrow \text{벡터 하나}$$

마지막에 격자를 펼쳐 벡터 하나로 뭉갠다. 출력은 “이미지 전체에 대한 답 하나”(예: 클래스 확률, 혹은 좌표 (x, y))다.

전결합층이 우리 문제에 맞지 않는 세 가지 이유

1. **파라미터 폭발.** $50 \times 50 \times 15 = 37,500$ 입력에서 $50 \times 50 = 2,500$ 출력을 내려면 가중치가 약 9,400만 개 — 3×3 합성곱 한 층(4,320개)의 2만 배다.
2. **공간 구조 상실.** 격자를 벡터로 펼치는 순간 “어느 칸이 어느 칸 옆인가”를 잊는다.
3. **입력 크기 고정.** $\mathbf{W}$ 의 크기가 n 에 묶여 격자 해상도를 바꾸면 층을 새로 만들어야 한다.

우리 논문에서는

전결합층은 벡터를 다루는 곳에만 쓴다 — 방어정 상태 $\mathbf{o}_p \in \mathbb{R}^8$ 를 질의 벡터로 바꾸는 $\text{MLP}_{8 \rightarrow 128 \rightarrow 32}$, 부화소 오프셋 머리 $\text{MLP}_{2d \rightarrow d \rightarrow 2}$, 그리고 아래 FiLM의 γ, β 생성기($64 \rightarrow 128$). 격자를 다루는 백본에는 전결합층이 없다.

3 완전 합성곱 신경망(FCN)

FCN(Fully Convolutional Network; Long, Shelhamer, Darrell, CVPR 2015)은 전결합층을 없애고 끝까지 합성곱(과 풀링·업샘플링)만 쓰는 신경망이다.

$$\text{이미지 } (H \times W) \rightarrow \text{conv} \rightarrow \text{conv} \rightarrow \cdots \rightarrow \text{conv} \rightarrow \text{출력 지도 } (H \times W)$$

출력이 벡터가 아니라 입력과 같은 격자 모양의 지도이고, 픽셀마다 값이 하나씩 붙는다. 원래 용도는 의미 분할(픽셀마다 “도로/사람/차”)이었다.

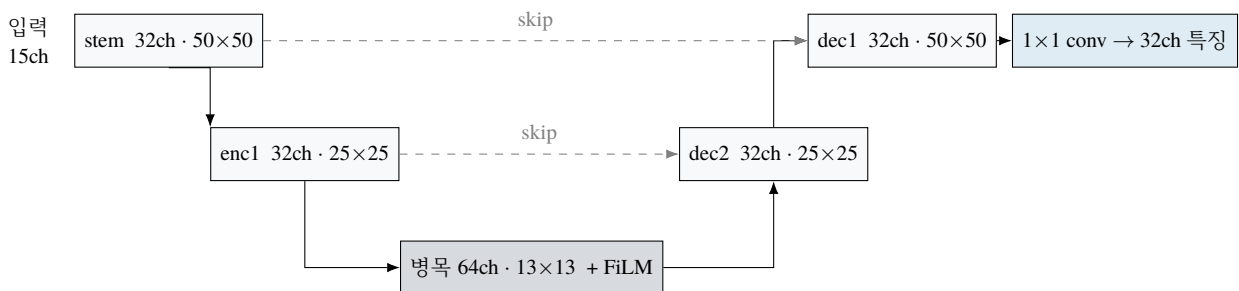
	보통 CNN(분류)	FCN
마지막 층	flatten → 전결합	합성곱
출력	벡터 하나(이미지 전체의 답)	픽셀마다 값 하나($H \times W$ 지도)
입력 크기	고정	임의
원래 용도	“이 사진은 고양이”	“이 픽셀은 도로, 저 픽셀은 사람”

합성곱만 쓰면 두 성질이 따라온다. 하나는 장점이고 하나는 다음 절의 문제가 된다.

- **가중치 공유**. 3×3 필터 하나가 격자 전체를 훑는다. 파라미터 수가 격자 크기와 무관하고, 선박 수가 바뀌어도 그대로 돈다.
- **평행이동 등변성(translation equivariance)**. 입력 무늬가 10칸 옆으로 옮겨가면 출력도 그대로 10칸 옆으로 옮겨간다. 뒤집어 말하면 합성곱 필터는 자기가 격자의 어디를 보고 있는지 모른다.

U-Net — FCN의 한 종류

U-Net(Ronneberger et al., 2015)은 FCN에 두 가지를 더한 구조다. (1) 인코더가 해상도를 줄여 수용 영역(한 픽셀이 참조하는 범위)을 넓히고 디코더가 원래상도로 되돌린다. (2) **skip 연결**이 인코더의 각 단계 출력을 같은 해상도의 디코더 단계에 그대로 이어 붙여, 축소 과정에서 흐려진 정밀한 위치 정보를 복원한다. 이름은 이 구조를 그리면 U자가 되어서다.



우리 논문에서는

정책 백본은 위 그림의 UNetLite(폭 32, 인코더 2단 $50 \rightarrow 25 \rightarrow 13$)다. 출력은 픽셀마다 32차원 특징 $\mathbf{f}_i$ 이고, 이것이 §4.4.2의 점수 $s_{p,i} = \langle \mathbf{W}_q \mathbf{q}_p + \mathbf{b}_q, \mathbf{f}_i \rangle / \sqrt{d}$ 의 재료다. 좌표 하나를 회귀하는 전결합 머리로는 유효 마스크를 칸 단위로 씌우거나 범주형 분포에서 칸을 뽑을 수 없으므로 (§4.4.3), 픽셀 지목 행동에는 FCN 계열이 필요조건이다.

4 CoordConv — “내가 어디에 있는가”

문제

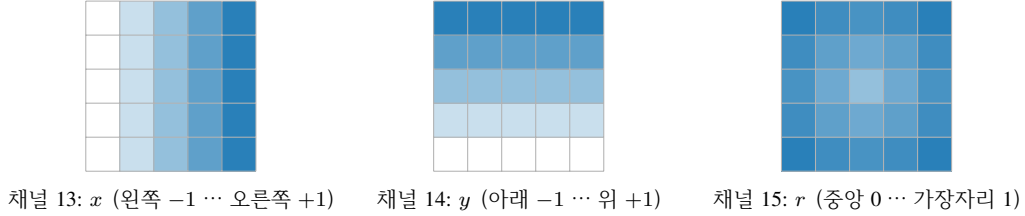
합성곱은 무늬만 보지 위치는 못 본다. 적선 밀집 덩어리가 격자 (10,10)에 있든 (40,40)에 있든 필터의 반응은 같다. 그런데 우리 문제에서는 절대 위치가 곧 의미다 — 모선은 항상 격자 중앙이고, 그물벽을 놓을 수 있는 자리는 “모선에서 반경 400–4,500 m 사이의 환상 구역”이며, “적과 모선 사이”라는 판단은 중앙을 기준으로 해야 성립한다.

순수 합성곱망이 이것을 하려면 “원점” 채널이나 “요격 구역” 채널의 경계 무늬를 여러 층에 걸쳐 전파해 간접적으로 위치를 추론해야 하는데, 이것이 의외로 잘 안 된다. Liu et al. (NeurIPS 2018, An

intriguing failing of convolutional neural networks and the CoordConv solution)은 “좌표 (i, j) 를 주면 그 자리에 점 하나를 찍은 이미지를 내라”는 아주 단순한 과제를 보통 CNN이 거의 풀지 못함을 보였다.

처방

입력에 좌표값 자체를 담은 채널을 덧붙인다. 그것이 전부다.



이러면 첫 합성곱부터 필터가 “지금 보는 칸의 x, y, r ”을 입력값으로 읽을 수 있으므로, “ $r \approx 0.5$ 인 칸은 좋다”, “적 덩어리보다 모션 쪽(r 이 작은 쪽)에 있는 칸은 좋다”처럼 위치에 의존하는 함수를 학습할 수 있다. 등변성이 필요한 부분(무늬 인식)은 공유 필터가 그대로 맡고, 위치가 필요한 판단만 좌표 채널을 읽어 한다.

우리 논문에서는

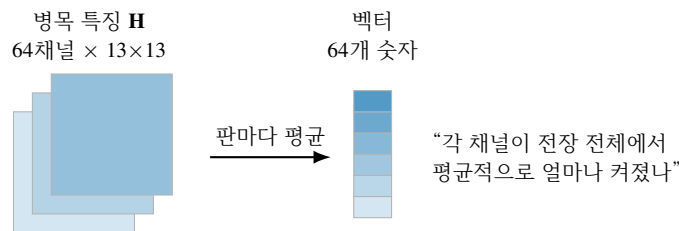
Table 4의 채널 13–15가 이것이다 — $(c_i^x, c_i^y) = (\mathbf{x}_i - \mathbf{m})/E$, $c_i^r = \text{clip}(\|\mathbf{x}_i - \mathbf{m}\|/E, 0, 1)$ (식 ch_coord). 코드에서는 설정에서 파생되는 상수 텐서라 학습 파라미터가 아니고 체크포인트에도 저장하지 않으며, U-Net 입구에서 한 번만 붙인다. 비용은 첫 conv의 입력 채널 3개분(가중치 288개, 전체의 0.1%)이다. r 채널을 따로 둔 이유는 x, y 에서 $\sqrt{x^2 + y^2}$ 을 합성하는 것이 원리상 가능해도 몇 층이 필요하고, 우리 문제의 핵심 기하(환상 요격 구역, 모션 거리)가 바로 r 이라 직접 주는 편이 싸기 때문이다.

5 GAP — 지도를 벡터로 요약하기

FiLM을 설명하기 전에 그 입력을 만드는 연산을 먼저 본다. 전역 평균 풀링(Global Average Pooling, GAP)은 채널마다 그 채널의 판($H \times W$) 전체를 평균 내어 숫자 하나로 만드는 연산이다.

$$\text{GAP}(\mathbf{H})_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W H_{c,i,j}$$

PyTorch 문서(`nn.AdaptiveAvgPool2d`, 2.13)는 이를 “입력 크기와 무관하게 출력 크기를 고정하며, 출력 채널 수는 입력 채널 수와 같다”고 적는다. 우리 코드의 `b.mean((2, 3))`은 이와 같은 연산이다.



왜 이것이 “전황 요약”인가. 병목의 각 채널은 학습된 어떤 패턴 검출기다 — 예컨대 어떤 채널은 “적 밀집 + 모션 근접”에, 다른 채널은 “이미 그물이 깔린 회랑”에 켜진다. 그 판을 평균 내면 “그런 상황이 전장 여기저기에 얼마나 많은가”가 되고, 64채널의 평균을 모으면 *위치 없이* 전장의 전체 상황을 요약한 벡터가 된다. 위치 정보는 일부러 버린다 — 위치별 정보는 특징 지도 자체와 skip 연결이 나르고 있고, 여기서 필요한 것은 “어디”가 아니라 “전체적으로 어떤가”이기 때문이다.

	창 크기	출력	용도
일반 평균/최대 풀링	2×2 등 작은 창	해상도를 줄인 지도	다운샘플링
전역 평균 풀링	판 전체	채널당 숫자 1개 → 벡터	지도 → 벡터 요약
전역 최대 풀링	판 전체	채널당 최댓값 1개	“어딘가에 있나” 검출

평균을 쓴 이유: 최대는 “한 곳이라도 강하게 켜졌나”만 보고, 평균은 “얼마나 널리 켜졌나”를 본다. 전황 요약에는 후자가 맞고, 기울기도 모든 픽셀에 고르게 흘러 학습이 안정적이다. 또한 출력 크기가 입력과 무관하게 고정되므로 격자를 30×30 으로 바꿔도 GAP 출력은 그대로 64차원이다.

6 FiLM — “전장 전체가 어떤 상황인가”를 모든 위치에 알리기

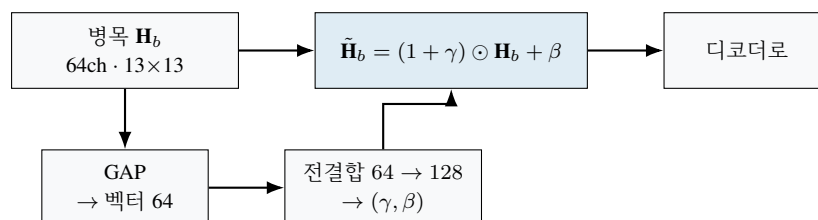
FiLM(Feature-wise Linear Modulation; Perez et al., AAAI 2018)은 어떤 조건 벡터로부터 배율 γ 와 이동 β 를 만들어, 특징 지도의 채널마다 “ γ 곱하고 β 더하기”를 해 주는 층이다.

$$\tilde{\mathbf{H}}_c = \gamma_c \cdot \mathbf{H}_c + \beta_c \quad (c = \text{채널 번호})$$

채널 64개면 γ 64개, β 64개 — 판마다 볼륨을 키우거나 줄이고(γ), 바닥을 올리거나 내린다(β). 픽셀 별로 다르게 하지 않고 *판 전체*를 똑같이 조절하는 것이 “feature-wise(채널별)”의 뜻이다.

원래 용도는 시각 질의응답이었다 — “빨간 공 왼쪽에 있는 물체는?”이라는 문장을 조건 벡터로 만들어 CNN의 채널을 변조하면, 같은 CNN이 질문에 따라 다른 것에 주목한다. 즉 “어떤 정보 X로 CNN의 동작을 바꾸고 싶다”할 때 X를 채널별 배율·이동으로 주입하는 범용 장치다. 새 채널을 붙이는 것보다 싸고(γ, β 는 벡터 하나), 영향이 전 위치에 균일하게 미친다.

우리 구현



여기서 조건 벡터가 외부 문장이 아니라 **망 자신의 병목을 GAP으로 평균 낸 벡터**라는 점이 우리 쓰임의 특징이다. 즉 “전장 전체를 한 번 훑은 요약”으로 “각 지점의 특징”을 조절한다.

왜 필요한가

- 합성곱은 국소적이다. 3×3 필터를 몇 층 쌓아도 각 픽셀은 주변 일부만 본다. 그런데 “그물벽을 어디에 칠까”는 전황에 달려 있다 — 적이 한 무리로 몰려오는지 셋으로 갈라졌는지, 이미 그물이 많이 깔렸는지, 전체가 급박한지.

- 이 전역 정보를 픽셀마다 고르게 전달하는 가장 싼 길이 FiLM이다. GAP 벡터가 “지금 전장은 이런 상황”을 담고, 그것이 $\gamma \cdot \beta$ 가 되어 모든 위치의 채널을 같은 방향으로 밀어 준다.
- $(1 + \gamma)$ 로 둔 이유: 학습 초기 $\gamma \approx 0, \beta \approx 0$ 이면 항등 — 처음엔 아무 변조도 하지 않는 상태에서 출발해 필요한 만큼만 배운다.

우리 논문에서는

식 (film) 그대로다. 파라미터는 전결합 하나($64 \rightarrow 128$, 약 8천 개)뿐이고 병목에서 한 번만 적용된다. 아래첨자 $F(\gamma_F, \beta_F)$ 는 FiLM을 뜻하며, 배정 유지 보너스 β (식 sticky)나 퍼텐셜 할인 γ 와는 무관하다.

7 포인터 네트워크와 어텐션 — “후보 중 하나를 지목”

지금까지의 도구는 모두 지각용이다 — 이미지를 픽셀마다 이해한다. 그런데 정책이 내야 하는 것은 “이해”가 아니라 “선택”이다. 후보마다 점수를 매겨 하나를 고르는 출력 형식은 다른 계열에서 왔다.

포인터 네트워크(Vinyals et al., 2015)는 출력이 고정된 어휘가 아니라 입력 원소 중 하나를 가리키는 인덱스인 신경망이다(예: 순회 외판원 문제에서 “다음에 방문할 도시”는 입력 도시 목록 중 하나). 각 후보 i 에 점수 s_i 를 매기고 softmax로 분포를 만든 뒤 하나를 뽑는다. 그 점수는 **어텐션(Vaswani et al., 2017)**과 같은 질의-후보 내적이다.

$$s_i = \frac{\langle \mathbf{q}, \mathbf{k}_i \rangle}{\sqrt{d}}, \quad \pi(i) = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

질의 $\mathbf{q}$ 는 “무엇을 찾는가”, 키 $\mathbf{k}_i$ 는 “후보 i 는 무엇인가”이고, 내적이 크면 잘 맞는 후보다.

우리 논문에서는

후보 i = 격자의 픽셀, 키 $\mathbf{k}_i$ = U-Net이 낸 픽셀 특징 $\mathbf{f}_i$, 질의 $\mathbf{q}_p$ = 방어정 p 의 자체 상태(위치·헤딩·그물 잔량·요격점)를 MLP로 바꾼 벡터. 따라서 식 (score) $s_{p,i} = \langle \mathbf{W}_q \mathbf{q}_p + \mathbf{b}_q, \mathbf{f}_i \rangle / \sqrt{d}$ 는 “방어정 p 가 픽셀 i 를 얼마나 원하는가”이고, 유효 마스크를 씌운 softmax(식 masked_softmax)가 곧 정책의 행동 분포다. 같은 특징 지도 위에서 방어정마다 질의만 달라 서로 다른 점수맵이 나온다.

8 합쳐서 읽는 §2.4

도구	합성곱이 못 하는 것	처방 / 우리 논문의 자리
전 결 합 층 제 거 (FCN)	픽셀마다 답 내기	전결합층 없이 격자 출력 → 점수맵의 전제
U-Net	넓은 문맥과 정밀 위치를 동시에	인코더-디코더 + skip → 백본 UNetLite
CoordConv	“내가 어디에 있는가”	좌표 x, y, r 을 입력 채널로 → 채널 13-15
GAP + FiLM	“전장 전체가 어떤 상황인가”	병목 평균 → 채널별 $\gamma, \beta \rightarrow$ 식 (film)
포인터/어텐션	후보 중 하나를 고르기	질의-픽셀 내적 → 식 (score), 행동 분포

§2.4의 논지는 이렇게 읽힌다.

1. FCN·U-Net·CoordConv·FiLM은 기존 연구에서 이미지를 이해하는 데(분할·인식) 쓰였다.

2. “여러 후보 중 하나를 고르는” 출력 형식은 포인터 네트워크·어텐션에서 왔고, 주로 조합 최적화에 쓰였다.
3. 본 연구의 새로움은 새 층을 발명한 것이 아니라, **지각용 조밀 출력을 선택용 포인터 점수로 읽어 그것을 그대로 강화학습의 행동 분포로 쓴 것이다**. 그래서 행동공간이 격자가 되고, 안전·임무 제약을 마스크로 분포에 직접 씌울 수 있으며(유효 집합 밖 확률이 정확히 0), 입출력 형태가 선박 수에 불변이다.

한 문장 요약

합성곱은 “무늬”를 보고(FCN/U-Net), 좌표 채널이 “위치”를 알려 주고(CoordConv), 병목 평균이 “전황”을 알려 주며(GAP→FiLM), 방어정의 질의가 그 위에서 “내가 갈 칸”을 가리킨다(포인터). 이 네 가지가 합쳐진 것이 점수맵 정책이다.

참고 문헌

- Long, Shelhamer, Darrell. Fully convolutional networks for semantic segmentation. CVPR 2015.
- Ronneberger, Fischer, Brox. U-Net: Convolutional networks for biomedical image segmentation. MICCAI 2015.
- Liu et al. An intriguing failing of convolutional neural networks and the CoordConv solution. NeurIPS 2018.
- Perez et al. FiLM: Visual reasoning with a general conditioning layer. AAAI 2018.
- Lin, Chen, Yan. Network in network. ICLR 2014. (전역 평균 풀링)
- Vinyals, Fortunato, Jaitly. Pointer networks. NeurIPS 2015.
- Vaswani et al. Attention is all you need. NeurIPS 2017.
- PyTorch 2.13 문서, `torch.nn.AdaptiveAvgPool2d`.